# TASK: 7

# CORS & HSTS

**Surendra Gabbani**

# CORS

## What is CORS ?

CORS means Cross-Origin Resource Sharing.

It is a security mechanism built into web browsers that controls how resources (like APIs, fonts, or data) can be requested from a different domain (origin) than the one currently being visited.

## What is an Origin?

- Origin = Protocol + Domain + Port
- Example: https://example.com:443 (protocol: https, domain: example.com, port: 443)
- Different origin example:

  http://example.com (different protocol)

  https://api.example.com (different subdomain)

## How CORS Works:

- Browser sends a request with **Origin header**
- Server responds with **Access-Control-Allow-Origin** header

Browser checks the response:

- If allowed → request succeeds
- If not allowed → request blocked

## Important Headers

- Access-Control-Allow-Origin → Which domains can access
- Access-Control-Allow-Methods → Allowed HTTP methods (GET, POST, etc.)
- Access-Control-Allow-Headers → Allowed custom headers
- Access-Control-Allow-Credentials → Whether cookies/auth can be shared

# HSTS

## What is HSTS ?

**HSTS (HTTP Strict Transport Security)** is a web security policy mechanism that ensures a website is always accessed using HTTPS instead of HTTP. When a server enables HSTS, it sends a special response header (Strict-Transport-Security) to the browser. The browser then remembers this rule and automatically upgrades all future requests to HTTPS, even if the user types or clicks on an http:// link.

## How HSTS Works

- Server sends Strict-Transport-Security header.
- Browser remembers to only use HTTPS for that domain.
- Even if the user types http://, browser upgrades to https://.

## Example Header:

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

- max-age: Time (in seconds) to enforce HTTPS.
- includeSubDomains: Applies rule to subdomains.
- preload: Eligible for browser HSTS preload list.

## Advantages:

- Protects against eavesdropping & tampering.
- No user intervention required.
- Reduces risk of session hijacking.

## Limitations:

- Initial connection might still be vulnerable.
- Requires proper SSL/TLS certificate management.
- Misconfiguration can lock out domains.

# LAB

## Lab: CORS vulnerability with basic origin reflection

APPRENTICE

LAB ✓ Solved

This website has an insecure CORS configuration in that it trusts all origins.

To solve the lab, craft some JavaScript that uses CORS to retrieve the administrator's API key and upload the code to your exploit server. The lab is solved when you successfully submit the administrator's API key.

You can log in to your own account using the following credentials: `wiener:peter`

ACCESS THE LAB

## STEP 1: (login with creditionals)

| | | | | | | |
|---|---|---|---|---|---|---|
| https://0a14000d042f7709... | GET | /academyLabHeader | | 101 | 147 | |
| https://0a14000d042f7709... | GET | /academyLabHeader | | 101 | 147 | |
| https://0a14000d042f7709... | GET | /accountDetails | | 200 | 303 | JSON |
| https://tags.srv.stackadap... | GET | /js_tracking?url=https%3A%2F%... | ✓ | 204 | 224 | |
| https://0a14000d042f7709... | GET | /login | | 200 | 3689 | HTML |

**Request**

Pretty Raw Hex

```
1 GET /accountDetails HTTP/2
2 Host: 0a14000d042f770980f667f4005d0066.web-security-academy.net
3 Cookie: session=DpU6WwKkbNV550meBcjH34lWVivsMF2I
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:142.0) Gecko/20100101 Firefox/142.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate
8 Referer: https://0a14000d042f770980f667f4005d0066.web-security-academy.net/my-account?id=wiener
9 Sec-Fetch-Dest: empty
0 Sec-Fetch-Mode: cors
1 Sec-Fetch-Site: same-origin
2 Priority: u=4
3 Te: trailers
4
```

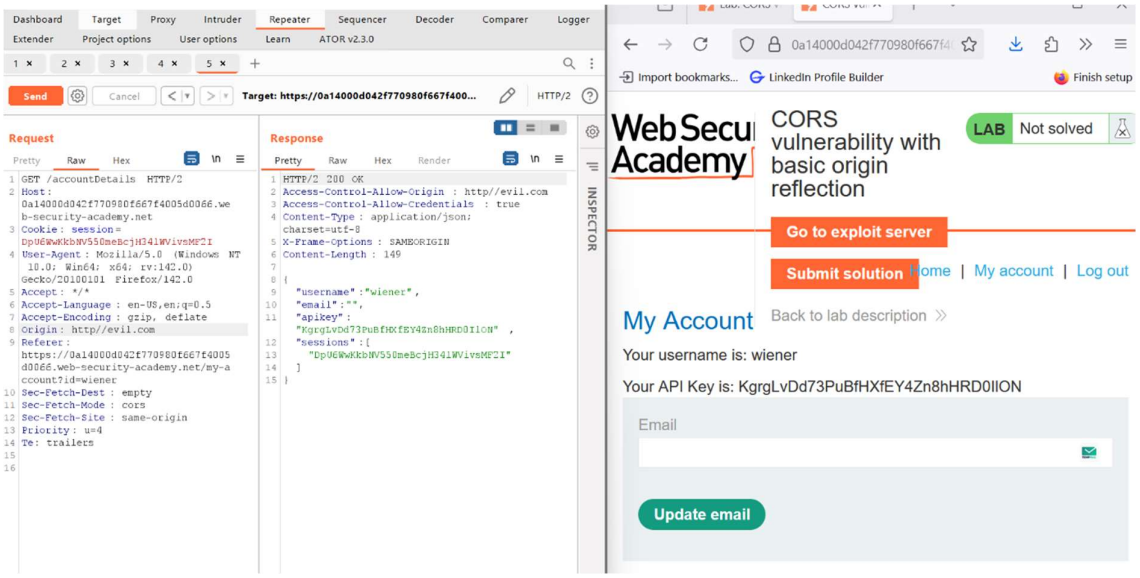
**Response**

Pretty Raw Hex Render

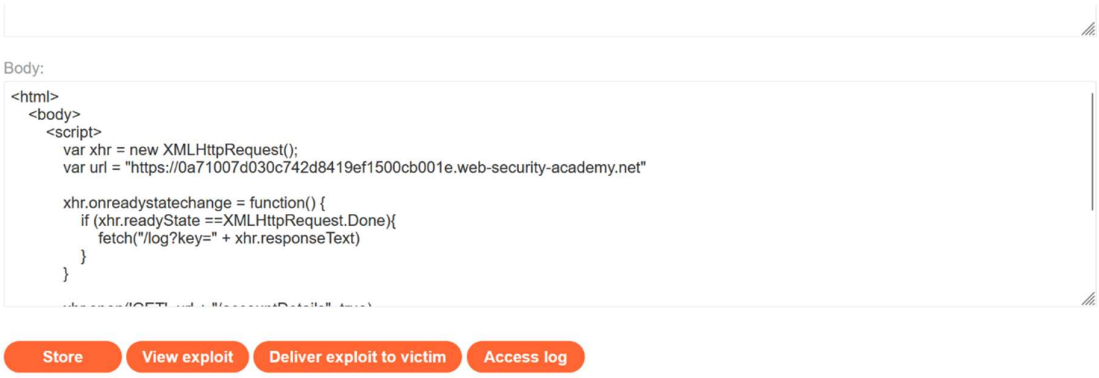
```
1 HTTP/2 200 OK
2 Access-Control-Allow-Credentials : true
3 Content-Type : application/json; charset=utf-8
4 X-Frame-Options : SAMEORIGIN
5 Content-Length : 149
6
7 {
8   "username" :"wiener",
9   "email" :"",
10  "apikey" : "KgrgLvDd73PuBfHXfEY4Zn8hHRD0IlON" ,
11  "sessions" :[
12    "DpU6WwKkbNV550meBcjH34lWVivsMF2I"
13  ]
14 }
```

INSPECTOR

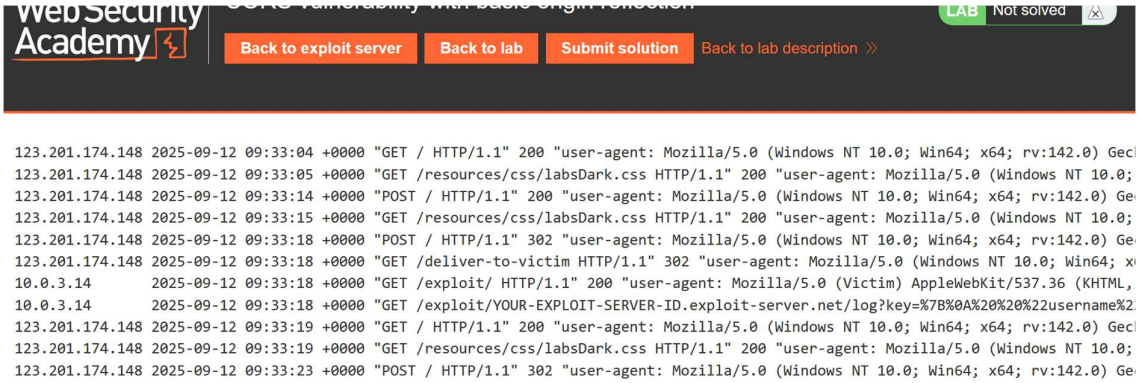
## STEP 2: (Send to repeater)



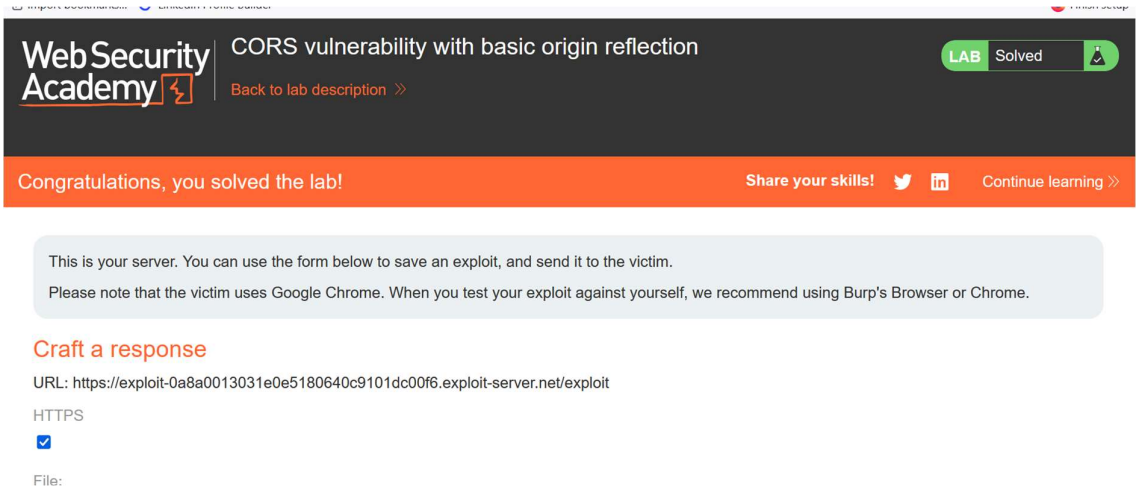

## STEP 3: (HTML Source code)




## STEP 4: (Go back to exploit server)



## STEP 5: Solved




## Security Concerns:

- Misconfigurations may cause data leaks
- Example: Access-Control-Allow-Origin: * with credentials enabled = risky
- Must carefully configure trusted origins only

## Conclusion:

- CORS enables safe cross-origin communication
- Protects users while allowing modern web applications to function
- Correct configuration = security + flexibility

# THANK YOU